



Идея артефактов.

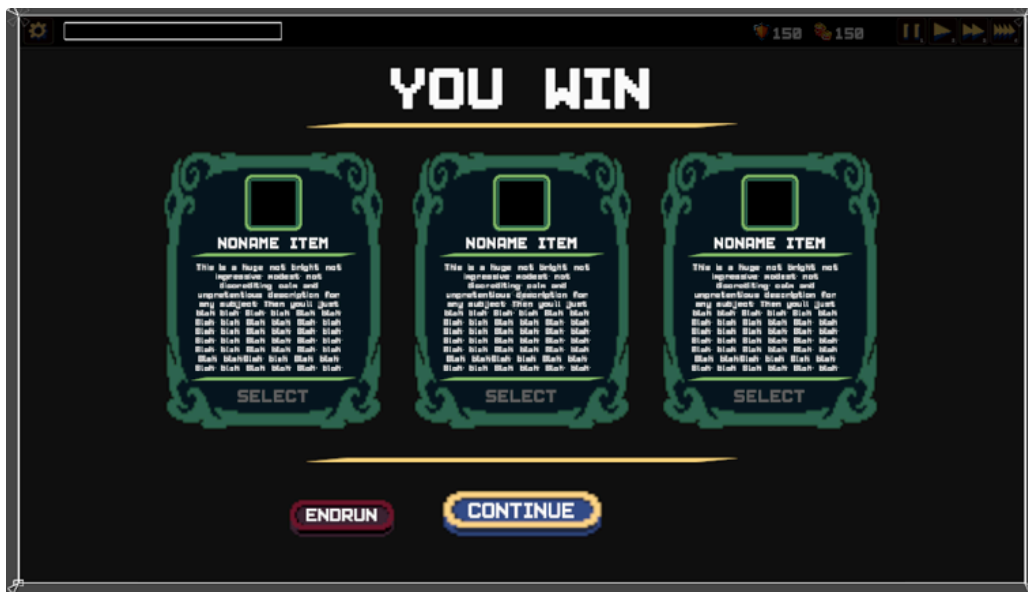
- В бесконечный TD хочется добавить что-то, что могло бы сделать игру веселее, при этом не позволяя игроку прогрессировать в пассивном плане. Если просто увеличивать силу мобов и силу игрока, то игра не будет становиться сложнее, ведь состояние сил всегда будет одинаково. Если позволять расти только силе врагов, то рано или поздно они станут очень сильными. Относительно простенькая система градации сложности предполагает, что игрок проиграет в любом случае, однако поражение (превосходство сил врага) можно оттянуть через специальные предметы. Вот только хотелось бы достичь такой специфики предметов, которая не просто бы улучшала характеристики, а делала игру разнообразнее и веселее, учитывая, что я всё-таки вряд ли смогу в хороший баланс.

Как и во всех рогаликах - предметы почти всегда падают именно после завершения этажа \ комнаты \ волны. Когда игрок преодалеет все волны и локация будет завершена, ему на выбор будет выдано три предмета, из которых он может взять лишь один.

NextFloorScreen (ArtefactsScreen).

- Если же игрок пережил все волны, то, в духе рогаликов, нам открывается меню с выбором одного из нескольких предметов. Здесь можно покинуть игру или продолжить. Однако не выбрав предмета - продолжить не получится.

Три бокса с описанием, иконкой и названием предмета - классика. При выборе предмета кнопка SELECT меняется на SELECTED и перекрашивается в зелёный. При выборе другого → прошлая возвращается в исходное состояние, а новая приобретает эффект выбора.



Интерфейс выбора предметов.



Выбранный предмет меняет надпись и подсвечиваемость.

Создаём класс под панели. Храним ссылки на все элементы панели, а также цвета текста. Создаем поле под ArtefactSO - ScriptableObject.

Initialize(ArtefactSO) - инициализируем наш интерфейс данными из пришедшего пакета данных.

Click() - вызывает событие клика. На родительском объекте панели висит IeventTrigger компонент, который обрабатывает клик и вызывает метод Click(). К слову, событие будет обрабатывать отдельный класс, он будет и вызывать SetSelected();

SetSelected() - эффект выбора панели. Меняет цвет и надпись, а также выключает затемнение.

GetArtefact() - возвращает сам SO артефакт. Понадобиться, когда игрок будет выбирать панель и нажимать продолжить, чтобы применить артефакт.

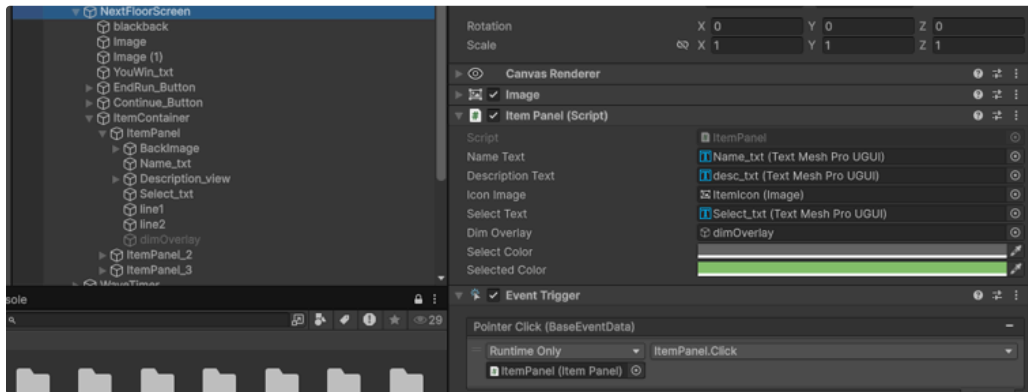
Clear() - сбрасывает интерфейс и поле артефакта, чтобы повесить следующие.

```
1 using System;
2 using TMPro;
3 using UnityEngine;
4 using UnityEngine.UI;
5
6 public class ItemPanel : MonoBehaviour
7 {
8     public Action<ItemPanel> OnClicked;
9
10    [SerializeField] private TMP_Text nameText;
11    [SerializeField] private TMP_Text descriptionText;
12    [SerializeField] private Image iconImage;
13    [SerializeField] private TMP_Text selectText;
14    [SerializeField] private GameObject dimOverlay;
15
16    [SerializeField] private Color selectColor;
17    [SerializeField] private Color selectedColor;
18
19    private ArtefactSO artefact;
20
21    public void Initialize(ArtefactSO data)
22    {
23        gameObject.SetActive(true);
24        artefact = data;
25        nameText.text = data.name;
26        descriptionText.text = data.description;
27        iconImage.sprite = data.icon;
28
29        SetSelected(false);
30        dimOverlay.SetActive(false);
31    }
32
33    public void Click()
34    {
35        OnClicked?.Invoke(this);
36    }
37
38    public void SetSelected(bool selected)
39    {
40        selectText.text = selected ? "SELECTED" : "SELECT";
41        selectText.color = selected ? selectedColor : selectColor;
42        dimOverlay.SetActive(!selected);
43    }
44
45    public ArtefactSO GetArtefact() => artefact;
46
47    public void Clear()
48    {
49        nameText.text = "";
50        descriptionText.text = "";
51        iconImage.sprite = null;
52        SetSelected(false);
53        selectText.text = "SELECT";
54        selectText.color = selectColor;
55        dimOverlay.SetActive(false);
```

```

56     gameObject.SetActive(false);
57 }
58
59 }

```



Иерархия и инспектор.

Создаём ScriptableObject класс, который будет конфигом для артефакта. А также сам эффект тоже делаем SO. Это позволит в артефакт пробрасывать SO настраиваемый, самостоятельный вариант эффекта.

```

1  using UnityEngine;
2
3  [CreateAssetMenu(menuName = "Artefacts/Artefact")]
4  public class ArtefactSO : ScriptableObject
5  {
6      public int IDA;
7      public string artefactName;
8      [TextArea] public string description;
9      public Sprite icon;
10
11     public ArtefactEffectSO effectArtefact;
12 }
13
14 public abstract class ArtefactEffectSO : ScriptableObject
15 {
16     public abstract void Equip();
17     public abstract void Dequip();
18 }

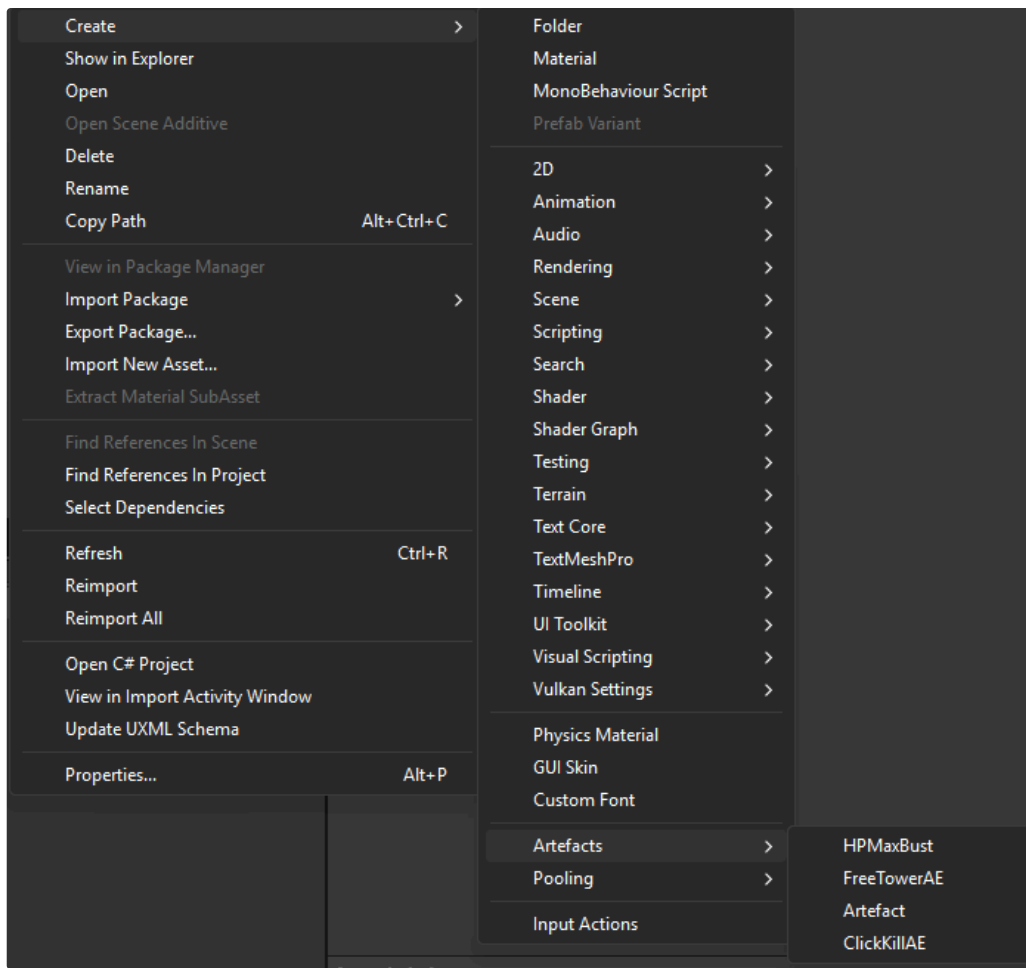
```

Пример предмета:

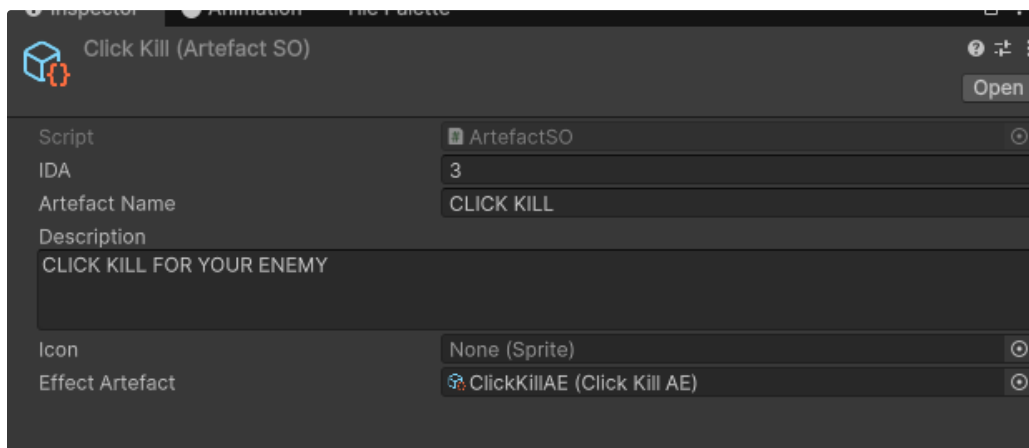
```

1  using UnityEngine;
2
3  [CreateAssetMenu(menuName = "Artefacts/ClickKillAE")]
4  public class ClickKillAE : ArtefactEffectSO
5  {
6      public override void Dequip()
7      {
8          Debug.LogError("Artefact 3 Dequip");
9      }
10
11     public override void Equip()
12     {
13         Debug.LogError("Artefact 3 Equip");
14     }
15 }

```



Создание SO эффекта.



Настройка конфига артефакта.

Теперь к самому скрипту экрана выбора предмета.

Храним лист панелей и выбранный артефакт. Скрываем сами себя и подписываем под событие каждой панели метод ClickedPanel();

Show() - предполагает включение и загрузку случайных предметов в панели. Мы просто будем пробрасывать полученные ArtefactSO внутрь ItemPanel.Initialize(ArtefactSO) и

выводить на экран.

```
1 using System;
2 using System.Collections.Generic;
3 using UnityEngine;
4 using UnityEngine.UI;
5
6 public class NextFloorScreen : MonoBehaviour
7 {
8     public Action OnEndRunWinMenu;
9     public Button continueButton;
10
11     [SerializeField] private List<ItemPanel> itemPanels;
12     [SerializeField] private int itemAmount = 3;
13     public int GetItemAmount() => itemAmount;
14     public void SetItemAmount(int amount) => itemAmount = amount;
15
16     private ItemPanel selectedPanel;
17     private ArtefactSO selectedArtefactSO;
18
19
20     private void Awake()
21     {
22         Hide();
23         foreach (var itemPanel in itemPanels)
24         {
25             itemPanel.OnClicked += ClickedPanel;
26         }
27     }
28
29     public void Show()
30     {
31         gameObject.SetActive(true);
32         LoadNewItems();
33     }
34
35     public void Hide() =>
36         gameObject.SetActive(false);
```

ContinueButton() - вызывается нажатием кнопки на интерфейсе. Передаёт в ArtefactManager выбранный предмет и вызывает продолжение игры у GameFlowManager.

Интересный момент - проверки на то, что предмет выбран - нет. Хотя и можно было обойтись флагом, решением стало просто сделать кнопку некликабельной. А как только выбирается хотя бы один предмет (клик по панели вызывает событие и событие вызывает ClickedPanel(передавая себя же)) - то кнопка становится активной. А вернуться к состоянию отсутствия выбора нельзя.

ClearPanels() - вызываем у каждой панели очистку интерфейса, устанавливаем все наши поля на null/false.

LoadNewItems() - очищаем панели, получаем список случайных предметов из менеджера, проверяем на меньшее значение. И инициализируем присутствующие панели. В случае если предметов будет меньше панелей, то панели не инициализируются и не включаются.

ClickedPanel(ItemPanel panel) - реагирует на событие и получает панель, по которой кликнули. Если панель и так выбрана - игнорируем. Если нет - выбираем панель, ищем среди листа панелей выбранную - Активируем и получаем её предмет.

```
1 public void ContinueButton()
2 {
3     if (continueButton == null) return;
4     ArtefactManager.Instance.EquipArtefact(selectedArtefactSO);
5     GameFlowManager.Instance.ContinueAfterArtifact();
6 }
7 public void EndRunButton() =>
8     GameFlowManager.Instance.Defeat();
9
10 public void LoadNewItems()
11 {
12     ClearPanels();
13     var items = ArtefactManager.Instance.GetRandomArtefacts(itemAmount);
14
15     int count = Mathf.Min(items.Count, itemPanels.Count);
16     if (count < 1) {
17         Debug.LogError("Not a single item was received from the ArtefactManager");
18     }
19     for (int i = 0; i < count; i++)
20     {
21         itemPanels[i].Initialize(items[i]);
22     }
23 }
24 private void ClearPanels()
25 {
26     foreach (var item in itemPanels)
27     {
28         item.Clear();
29     }
30     selectedPanel = null;
31     selectedArtefactSO = null;
32     continueButton.interactable = false;
33 }
34
35 // Метод не очищает панели как объекты, поэтому отписки пока не тре
36 // Панели по стандарту будет три. Для игры этого может быть достато
37 // Однако, логику можно будет слегка изменить, чтобы допустить возм
38 // динамически, по ходу забега. В таком случае отписки следует
39
40
41 private void ClickedPanel(ItemPanel panel)
42 {
43     if (panel == selectedPanel) { return; }
44     selectedPanel = panel;
45     continueButton.interactable = true;
46
47     foreach (var item in itemPanels)
48     {
49         if (selectedPanel == item)
50         {
51             selectedPanel.SetSelected(true);
52             selectedArtefactSO = selectedPanel.GetArtefact();
53         }
54         else item.SetSelected(false);
55     }
56 }
57
58 }
```

Сейчас NextFloorScreen работает с панелями по ссылкам из инспектора. А количество предметов жестко выставлено в три. Для данного проекта этого достаточно и он не предполагает наличие большего количества. Однако, расширить систему можно очень легко, немного изменив NextFloorScreen, создавая нужное количество панелей или просто переиспользуя большее количество, чем 3. (Даже сейчас можно указать в лист больше панелей, создав их на сцене, просто использоваться будет только 3. Можно будет в таком случае указать случайный дроп 2-5 предметов на выбор, но для меня это не нужно).

ArtefactsManager.

- ❗ Нам нужен некий менеджер, который будет хранить все артефакты игры и выдавать случайные по запросу. Он же послужит своеобразным инвентарем.

Храним лист ArtefactSO - все доступные предметы.

HashSet<ArtefactSO> - храним все надетые артефакты.

GetRandomArtefacts(int) - сверяем все и те, что уже надеты, собираем из доступных лист и выкидываем наружу.

EquipArtefact(ArtefactSO) - принимает артефакт, запоминает, что он надет, а также вызывает абстрактный метод экипировки, что эффект предмета начал действовать.

DequipArtefact(ArtefactSO) - отключаем эффект, достаём из хэш сета объект.

Reset() - сбрасывает все эффекты и чистит коллекцию.

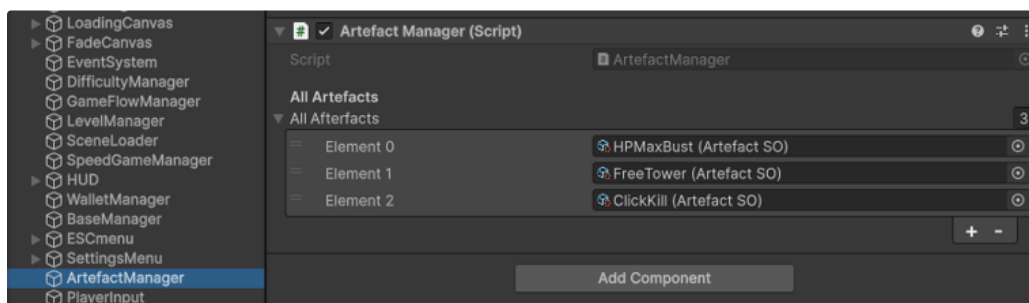
```
1 using NUnit.Framework;
2 using System.Collections.Generic;
3 using System.Linq;
4 using UnityEngine;
5
6 public class ArtefactManager : Manager<ArtefactManager>
7 {
8     [Header("All Artefacts")]
9     [SerializeField] private List<ArtefactSO> allArtefacts;
10
11     private HashSet<ArtefactSO> equippedArtefacts = new();
12
13     public List<ArtefactSO> GetRandomArtefacts(int count)
14     {
15         List<ArtefactSO> available = new();
16         foreach (var artefact in allArtefacts)
17         {
18             if (!equippedArtefacts.Contains(artefact))
19             {
20                 available.Add(artefact);
21             }
22         }
23     }
24 }
```



```

24         List<ArtefactSO> result = new();
25
26         count = Mathf.Min(count, available.Count);
27
28         while(result.Count < count)
29         {
30             int index = Random.Range(0, available.Count);
31             result.Add(available[index]);
32             available.RemoveAt(index);
33         }
34
35         return result;
36     }
37
38     public void EquipArtefact(ArtefactSO artefact)
39     {
40         if (!equippedArtefacts.Contains(artefact))
41         {
42             equippedArtefacts.Add(artefact);
43             artefact.effectArtefact.Equip();
44         }
45     }
46
47     public void DequipArtefact(ArtefactSO artefact)
48     {
49         if (equippedArtefacts.Contains(artefact))
50         {
51             equippedArtefacts.Remove(artefact);
52             artefact.effectArtefact.Dequip();
53         }
54     }
55
56     public void ResetInventory()
57     {
58         foreach (var item in equippedArtefacts.ToList())
59         {
60             DequipArtefact(item);
61         }
62         equippedArtefacts.Clear();
63     }
64 }

```



Ручное хранение всех артефактов.

Addressables.

Лично мне, в случае одиночной разработки пет проекта хватило бы и обычной системы ручного хранения, но хорошо бы понимать, как обстоят дела у серьёзных людей. А лучше даже внедрить!

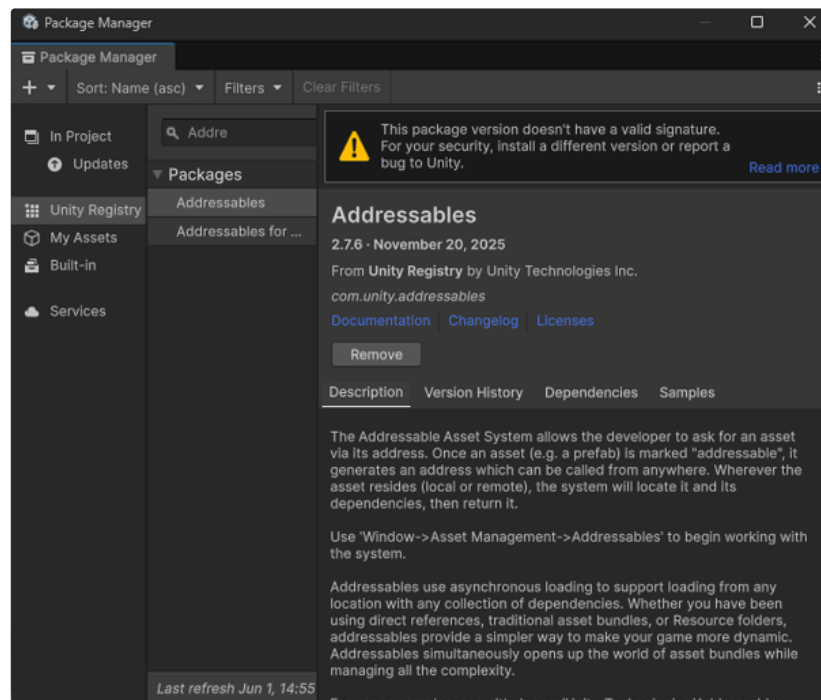
В небольших проектах многие зависимости можно настроить вручную через Inspector. Однако при росте количества ресурсов становится сложнее контролировать их загрузку, зависимости и использование памяти.

Когда речь идёт о большом количестве SO, префабов, моделей, текстур и других ресурсов, обычно используют специализированные механизмы загрузки ресурсов.

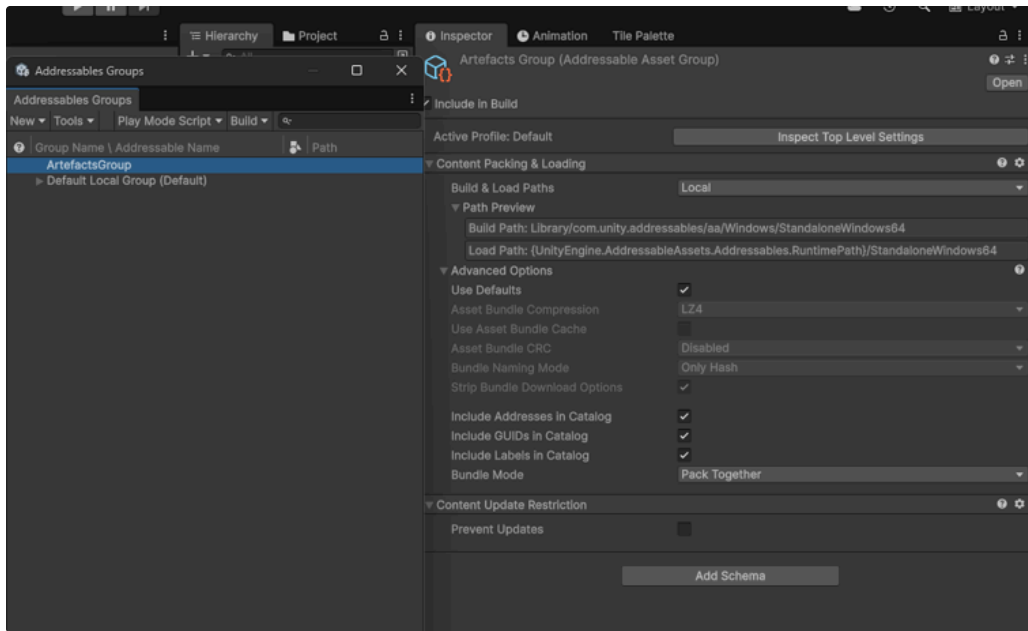
Одним из самых старых решений является папка Resources. Например, можно поместить все ArtefactSO в каталог: Assets/Resources/Artefacts/
А затем просто подгружать следующим образом: `allAfterfacts = Resources.LoadAll<ArtefactSO>("Artefacts").ToList();`

Однако содержимое Resources всегда включается в билд. Кроме того, при запуске проекта Unity создаёт индекс всех ресурсов из этой папки, что увеличивает длительность загрузки и потребление памяти.

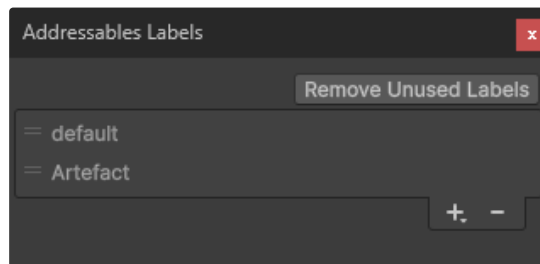
- ✓ Более современным решением является система Addressables, позволяющая адресно загружать и выгружать ресурсы по мере необходимости.



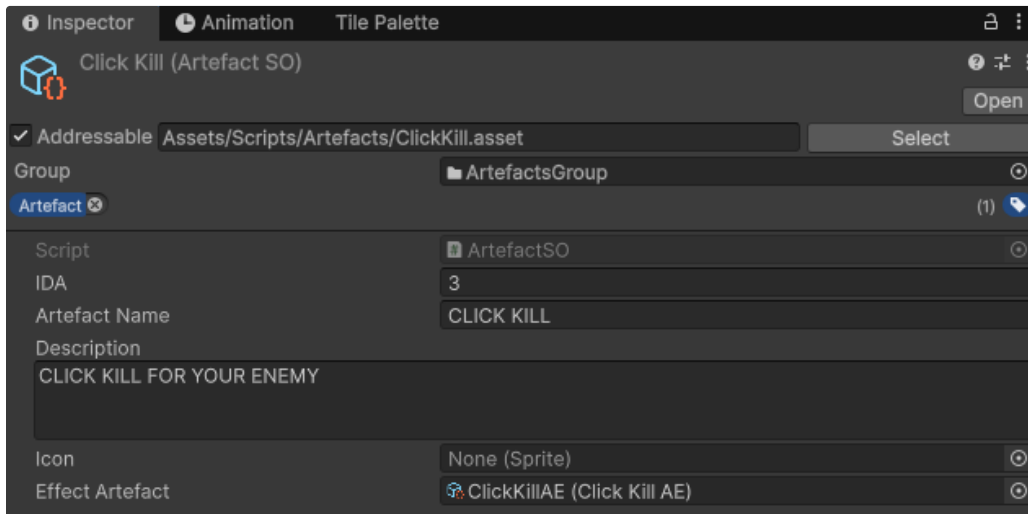
Устанавливаем из менеджера пакетов.



Создаём свою группу для артефактов.



Создаём специальную метку.



Устанавливаем галочку Addressable, группу и метку.

Теперь меняем ручное заполнение на следующий вариант:

Подключаем пространство имён. Создаём асинхронную инициализацию. Получаем операцию загрузки ассетов по типу `ArtefactSO`, передав туда метку `Artefact`, в поле `handle`.

Ожидаем когда оно выполнится. После проверяем статус готовности и заполняем полученными результатами операции наш лист. Готово!

```
1 using UnityEngine.AddressableAssets;
2
3 public class ArtefactManager : Manager<ArtefactManager>
4 {
5     private List<ArtefactSO> allArtefacts = new();
6     private HashSet<ArtefactSO> equippedArtefacts = new();
7
8     protected override void Awake()
9     {
10         base.Awake();
11         InitializeAllArtefacts();
12         if (allArtefacts.Count < 1)
13             Debug.LogError("Artefacts not detected!");
14     }
15
16     public async void InitializeAllArtefacts()
17     {
18         var handle = Addressables.LoadAssetsAsync<ArtefactSO>("Artefact
19         await handle.Task;
20         if (handle.Status == UnityEngine.ResourceManagement.AsyncOperat
21         {
22             allArtefacts = new List<ArtefactSO>(handle.Result);
23         }
24     }
```

- ✓ Addressables позволяет работать на оптимизацию. Он даёт контроль над памятью, зависимостями и моментов загрузки ресурсов.

При использовании менеджером прямых ссылок на ресурсы, Unity вынужден учитывать все зависимости этих объектов. Прямая ссылка сама по себе не означает, что всё будет инстанцировано в сцене, но гарантирует, что Unity будет учитывать их при построении зависимостей и упаковке билда.

При Resources - Unity создаёт индекс всех ресурсов, находящихся в папке. При запуске игры движок должен построить индекс всех объектов, что увеличивает время старта и потребление памяти. При Resources.Load(...) ресурс загружается в память и становится доступным. Resources.UnloadUnusedAssets() - должен удалить объект, но он сможет это сделать лишь в случае, если на объект не осталось ссылок. В противном случае объект останется в памяти, так как на него всё ещё существуют ссылки.

Addressables заменяет прямые ссылки на косвенные (AssetReferences). AssetReference хранит GUID и информацию о расположении ресурса. Например, мы храним некоторые объекты локации отдельной меткой. На экране загрузки на эту локацию мы вызовем `await Addressables.LoadAssetsAsync<GameObject>("City2_Content", null).Task;` При такой загрузке по метке Addressables самостоятельно определяет, какие AssetBundle необходимо загрузить. Когда же

игрок покидает локацию и переходит на новую, мы можем выгрузить данные из памяти: `Addressables.Release(city2Handle)`; Однако и это не гарантирует немедленную выгрузку. Команда `Release` уменьшает счётчик ссылок на загруженный ресурс. Когда же он перестанет кем-либо использоваться, то `Addressables` сможет выгрузить его из памяти.

- `Addressables` строится поверх `AssetBundle`. Во время билда `Addressables` автоматически собирает помеченные ресурсы в `AssetBundle` (.bundle) и создаёт каталог адресов для их поиска и загрузки. `Addressables` строится поверх `AssetBundle` и автоматизирует работу с ними. Например, DLC может быть собрано в отдельный `AssetBundle` и загружаться с сервера только после покупки или активации дополнения.

Update:

- Представим, что игру сломали и теперь можно пройти больше этажей, чем я мог рассчитывать возможным. Можно добавить панель YOU WIN, в качестве уважения. Однако игра всё-таки на рекорд, поэтому заканчивать игру совсем - не хочется. Предметы рано или поздно закончатся, и игра сломается. На такой случай я добавлю список не слишком оригинальных предметов, которые немного повышают характеристики, но которые могут складываться и попадаться игроку, даже когда уже попадались.